SmartPizza AI – Intelligent Food Ordering & Delivery Optimization System

Business Documentation

1. Problem Statement

The food delivery industry has experienced rapid growth due to increasing customer demand for convenience, faster service, and digital ordering experiences. However, many traditional food ordering platforms still rely on generic recommendations, static menus, limited customer personalization, and basic delivery tracking mechanisms. As a result, customers often spend more time selecting food items, miss relevant offers, and receive limited visibility into the status of their orders after placement.

In addition, food businesses face challenges in managing customer preferences, increasing repeat purchases, optimizing delivery operations, analyzing sales performance, and providing targeted promotions. Without intelligent recommendation systems and business analytics, restaurants may lose opportunities to improve customer satisfaction and maximize revenue.

To address these challenges, **SmartPizza AI – Intelligent Food Ordering & Delivery Optimization System** is developed as a comprehensive web-based platform that combines food ordering, AI-powered recommendations, delivery tracking, online payments, and business analytics into a single integrated solution. The system enables customers to browse pizzas, receive personalized recommendations based on previous orders and preferences, apply coupons, place orders securely, and track deliveries in real time.

The application also provides administrators with tools to manage pizzas, combos, coupons, users, orders, and payments while monitoring business performance through analytics dashboards. Delivery personnel can efficiently manage assigned deliveries and update order status throughout the delivery lifecycle.

By integrating modern technologies such as **React, Spring Boot, MySQL, JWT Authentication, Razorpay Payment Gateway, and AI-based Recommendation Logic**, SmartPizza AI aims to enhance customer experience, improve operational efficiency, streamline delivery management, and support data-driven decision-making for business growth.

2. Objectives

The objective of SmartPizza AI is to develop an intelligent pizza ordering and delivery platform that provides secure ordering, AI-based recommendations, online payments, and real-time delivery tracking. The system also helps administrators manage business operations efficiently through analytics and reporting.

3. User Stories

Customers can browse pizzas, place orders, make payments, and track deliveries. Admins can manage products, orders, and business analytics. Delivery partners can view assigned orders and update delivery status.

4. Use Cases

The system supports user registration, login, pizza ordering, cart management, coupon application, payment processing, order tracking, delivery management, and analytics monitoring.

5. Scope

The project covers authentication, ordering, payments, AI recommendations, delivery tracking, and admin analytics. Advanced features such as drone delivery and voice ordering are excluded from the current scope.

Technologies Used

Frontend Technologies

Technical Documentation

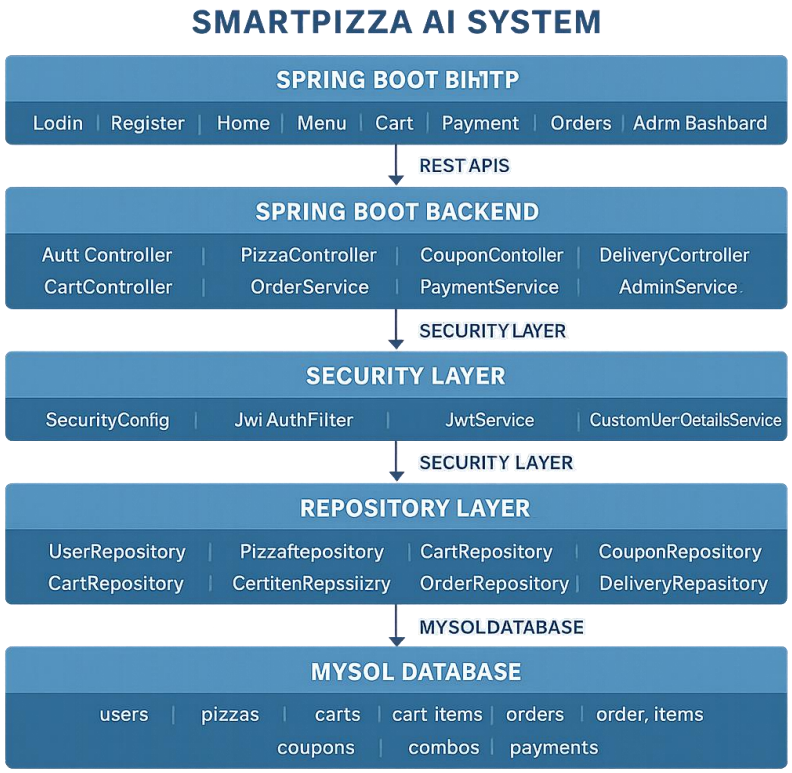
Technology	Purpose
React JS	Building interactive user interfaces
React Router DOM	Page navigation and routing
Axios	API communication with backend
HTML5	Structure of web pages
CSS3	Styling and responsive design
JavaScript (ES6+)	Client-side programming
Chart.js	Admin analytics charts and reports
React Icons	UI icons and visual enhancements
Bootstrap/CSS Flexbox	Responsive layouts and components

Backend Technologies

Technology	Purpose
Java 17	Core programming language
Spring Boot	Backend application development
Spring Security	Authentication and authorization
JWT (JSON Web Token)	Secure user login and role management
Spring Data JPA	Database interaction and ORM
Hibernate	Object Relational Mapping
Maven	Dependency management and build tool
REST APIs	Communication between frontend and backend
Swagger	API documentation and testing
Lombok	Reduces boilerplate code
Jakarta Validation	Input validation

1. Architecture Diagram

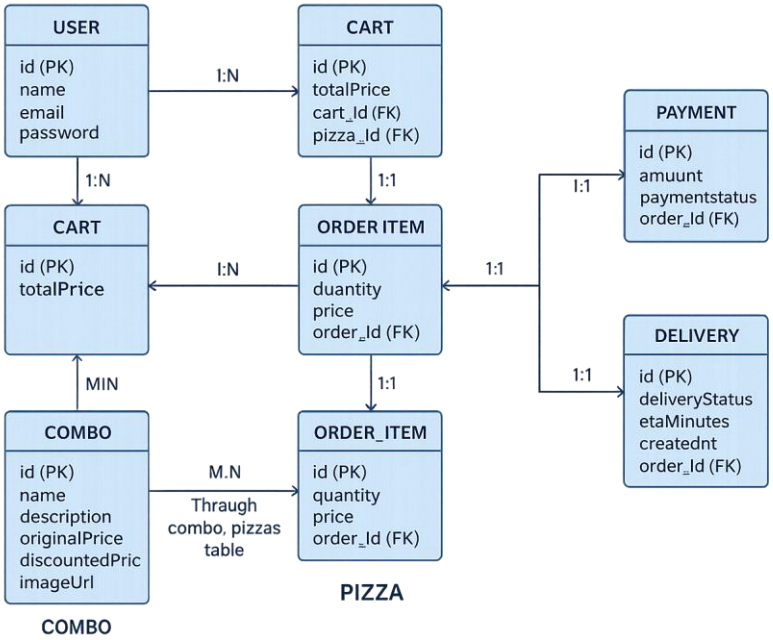
The system follows a three-tier architecture consisting of a React frontend, Spring Boot backend, and MySQL database. Communication between frontend and backend is handled through REST APIs secured with JWT authentication.



2. Database Design

The SmartPizza AI database is designed using a relational model to efficiently manage users, pizzas, orders, payments, deliveries, coupons, and combo offers. A user can maintain one cart and place multiple orders. Each order contains one or more order items and is associated with a payment record and delivery details. The cart stores selected pizzas before checkout through cart items. Combo offers are implemented using a many-to-many relationship with pizzas through a junction table (combo_pizzas). This design ensures data consistency, eliminates redundancy, and supports efficient retrieval of customer, order, and delivery information.

Relationship Summary



Parent Entity	Child Entity	Relationship Type
User	Cart	One-to-One
User	Order	One-to-Many
Cart	CartItem	One-to-Many
Pizza	CartItem	One-to-Many
Order	OrderItem	One-to-Many
Pizza	OrderItem	One-to-Many
Order	Payment	One-to-One
Order	Delivery	One-to-One
Combo	Pizza	Many-to-Many

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/smartpizza
spring.datasource.username=root
spring.datasource.password=Jama@24122003

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

server.port=8080

logging.level.root=ERROR

jwt.secret=smartpizzajwtsecretkeysmartpizzajwtsecretkey
```

```
mysql> use smartpizza;
Database changed
mysql> show tables;
+-----------------------+
| Tables_in_smartpizza  |
+-----------------------+
| cart                  |
| cart_item             |
| combo                 |
| combo_pizza           |
| coupon                |
| delivery              |
| order_item            |
| orders                |
| payment               |
| pizza                 |
| users                 |
+-----------------------+
11 rows in set (0.00 sec)
```

```
mysql> select * from users;
+----+-----------------+-----------+--------------------------------------------------------------+----------+---------+
| id | email           | name      | password                                                     | role     | cart_id |
+----+-----------------+-----------+--------------------------------------------------------------+----------+---------+
|  1 | bhavya@gmail.com | bhavya    | $2a$10$WssOeS31gwB8LhC4IoeAY.oaFIdyXYW.zx3ywQxL.U6hnM7xXflPa | CUSTOMER |       1 |
|  2 | sonu@gmail.com   | Sonu      | $2a$10$H50epQTcp7rXBuLa8kWJqueWrU9zgQdUWN8xPWktt3JKk04R7zaXa | ADMIN    |       2 |
|  3 | anu@gmail.com    | anu       | $2a$10$MUIjz2cFDMvBfcdHtDoaYOR3IUGue0GVbtnklYicM9EHdsyxQ0AeC | ADMIN    |       8 |
|  4 | sai@gmail.com    | sampreeth | $2a$10$Yedx2P8KBsu9HGwcQL0t4O645Q8mzk8GMv9PsUw/y99gXgbxJnBju | DELIVERY |       9 |
|  5 | ammu@gmail.com   | ammu      | $2a$10$7Wz0kTWjUzzvXs7f9jrRFe1q6QSeAoynrxKXm3xMYdRArIrSs3AvO | ADMIN    |      10 |
|  6 | sam@gmail.com    | sam       | $2a$10$ozatWTjfTcPj6dhJ0MCqVej0XV4hGHkzOK0v5lq5BldhdqOeb86G2 | ADMIN    |      11 |
|  7 | mani@gmail.com   | mani      | $2a$10$ZHWwSBrNQgVG37ahyPJYHuM1qxOB37kYuKzUVvl/N7V62mGo7hTKG | DELIVERY |      12 |
|  8 | pooji@gmail.com  | pooji     | $2a$10$sz6HKd/LdzHD1ns7JcRbiuDF5ixFpbeSTZID4zu9OMfY/HP169fkW | CUSTOMER |      13 |
|  9 | varsha@gmail.com | varsha    | $2a$10$hlDAulclz.D/8/TtBWIzAOA1D1jeyOPK.43B5TD19pxIvJhve.1CC | CUSTOMER |      14 |
| 10 | kittu@gmail.com  | kittu     | $2a$10$iduIEoUZW4n.U9a7BjzNUOr9tItjqk/2.HpyxnJd2TL9w3VRRBu0G | CUSTOMER |      15 |
| 11 | manu@gmail.com   | manu      | $2a$10$0HF7Oy1jd8SZeJYe5H/isur0LVCSp.qhSQKUdF51C24F0TX0RPipK | CUSTOMER |      16 |
+----+-----------------+-----------+--------------------------------------------------------------+----------+---------+
11 rows in set (0.01 sec)
```

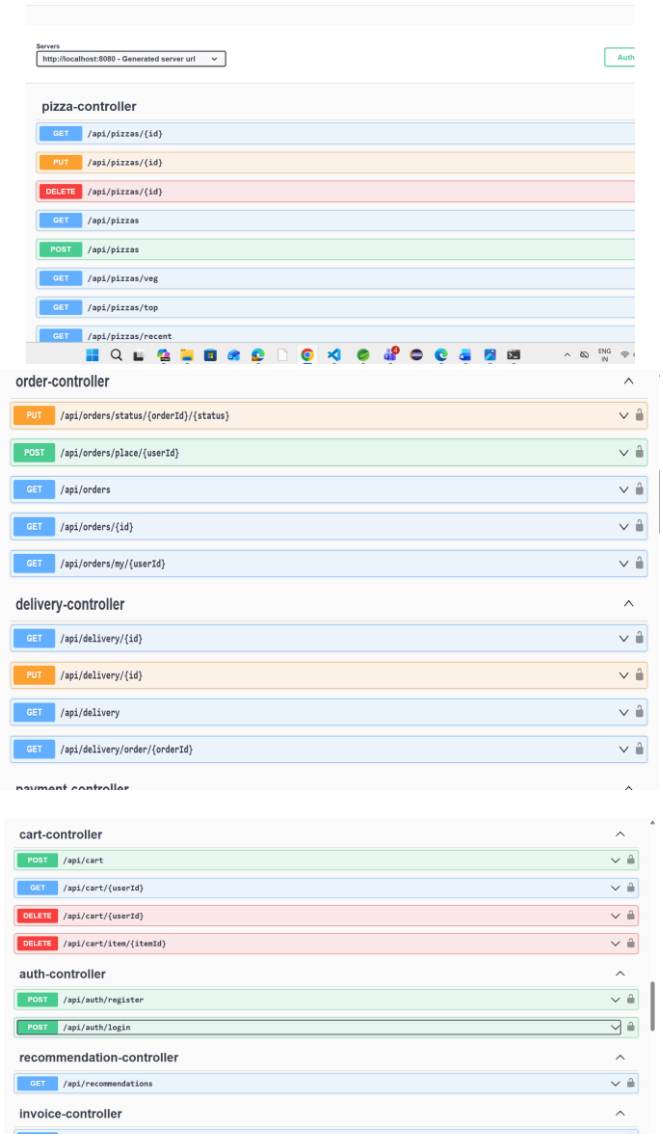
```
mysql> select * from pizza;
+----+---------------------+----------+--------------------------------+----------------------+-------+
| id | available           | category | description                    | name                 | price |
+----+---------------------+----------+--------------------------------+----------------------+-------+
|  1 | 0x01                | VEG      | Cheese loaded classic pizza    | Classic Cheese Pizza |   299 |
|  2 | 0x01                | VEG      | Loaded with vegetables         | Veg Supreme          |   299 |
|  3 | 0x01                | VEG      | Paneer  Loaded                 | Paneer Pizza         |   399 |
|  4 | 0x01                | NON-VEG  | Loaded with chicken            | Chicken Pizza        |   499 |
|  6 | 0x01                | VEG      | Loaded with vegetables         | Veg Pizza            |   250 |
|  7 | 0x00                | VEG      | Loaded with Corn and Vegetables | Corn pizza           |   350 |
+----+---------------------+----------+--------------------------------+----------------------+-------+
6 rows in set (0.01 sec)

mysql>
```

```
mysql> desc pizza;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| id          | bigint       | NO   | PRI | NULL    | auto_increment |
| available   | bit(1)       | YES  |     | NULL    |                |
| category    | varchar(255) | YES  |     | NULL    |                |
| description | varchar(255) | YES  |     | NULL    |                |
| name        | varchar(255) | YES  |     | NULL    |                |
| price       | double       | YES  |     | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
6 rows in set (0.01 sec)
```

```
mysql> select * from coupon;
+----+----------------+---------+----------+------------+
| id | active         | code    | discount | min_amount |
+----+----------------+---------+----------+------------+
|  1 | 0x01           | SAVE10  |       10 |        200 |
|  2 | 0x01           | SAVE20  |       20 |        500 |
|  3 | 0x01           | FIRST50 |       50 |       1000 |
+----+----------------+---------+----------+------------+
3 rows in set (0.00 sec)
```

3 .API Documentation

The SmartPizza AI application uses REST APIs to enable communication between the React frontend and Spring Boot backend. These APIs handle authentication, pizza management, cart operations, order processing, payment integration, delivery tracking, coupon management, and AI recommendations.



4.Deployment Guide

The React application is deployed on a web server, while the Spring Boot application runs on an application server connected to a MySQL database. Environment configurations are managed through application properties.

5. Folder Structure

The project follows a layered architecture with separate folders for components, pages, services, controllers, repositories, entities, security, and configurations. This structure improves maintainability and scalability.

src/main/java

├── com.wipro.pizzaapp

├── config

├── controller

├── dto

├── entity

├── exception

├── factory

├── repository

├── security

├── service

├── serviceimpl

└── strategy

Main Application Package

com.wipro.pizzaapp

Contains the main Spring Boot application class (PizzaappApplication.java) responsible for starting and bootstrapping the application.

Configuration Package

config

Contains application configuration classes.

- **SecurityConfig** – Configures Spring Security and JWT authorization.
- **SwaggerConfig** – Configures Swagger API documentation.
- **WebConfig** – Handles CORS and web-related configurations.

Controller Package

controller

Contains REST API controllers that handle incoming HTTP requests and responses.

Key Controllers:

- AuthController
- PizzaController
- CartController
- OrderController
- PaymentController
- CouponController
- ComboController
- DeliveryController
- RecommendationController
- InvoiceController
- AdminController

DTO Package

dto

Contains Data Transfer Objects (DTOs) used to transfer data between frontend and backend.

Examples:

- LoginRequest
- RegisterRequest
- AuthResponse
- PizzaDTO
- CartDTO
- OrderDTO
- PaymentDTO

- DeliveryDTO

Entity Package

entity

Contains JPA entity classes representing database tables.

Main Entities:

- User
- Pizza
- Cart
- CartItem
- Order
- OrderItem
- Payment
- Coupon
- Combo
- Delivery
- Role

Exception Package

exception

Provides centralized exception handling and custom exceptions for better error management.

Classes:

- GlobalExceptionHandler
- ResourceNotFoundException
- DuplicateResourceException

- UnauthorizedException

Factory Package

factory

Implements the Factory Design Pattern.

- **PaymentFactory** creates appropriate payment objects and strategies dynamically based on the selected payment method.

Repository Package

repository

Contains Spring Data JPA repositories used for database operations.

Examples:

- UserRepository
- PizzaRepository
- CartRepository
- OrderRepository
- PaymentRepository
- CouponRepository
- DeliveryRepository

These repositories interact directly with the MySQL database.

Security Package

security

Contains JWT-based authentication and authorization components.

Classes:

- JwtService

- JwtAuthFilter
- CustomUserDetailsService

  This package secures APIs and enforces role-based access control.

Service Package

service

Contains service interfaces that define business operations.

Examples:

- AuthService
- UserService
- PizzaService
- CartService
- OrderService
- PaymentService
- DeliveryService
- RecommendationService
- AdminService

Service Implementation Package

serviceimpl

Contains implementations of service interfaces where the actual business logic is written.

Examples:

- AuthServiceImpl
- PizzaServiceImpl
- CartServiceImpl

- OrderServiceImpl
- PaymentServiceImpl
- RecommendationServiceImpl

Strategy Package

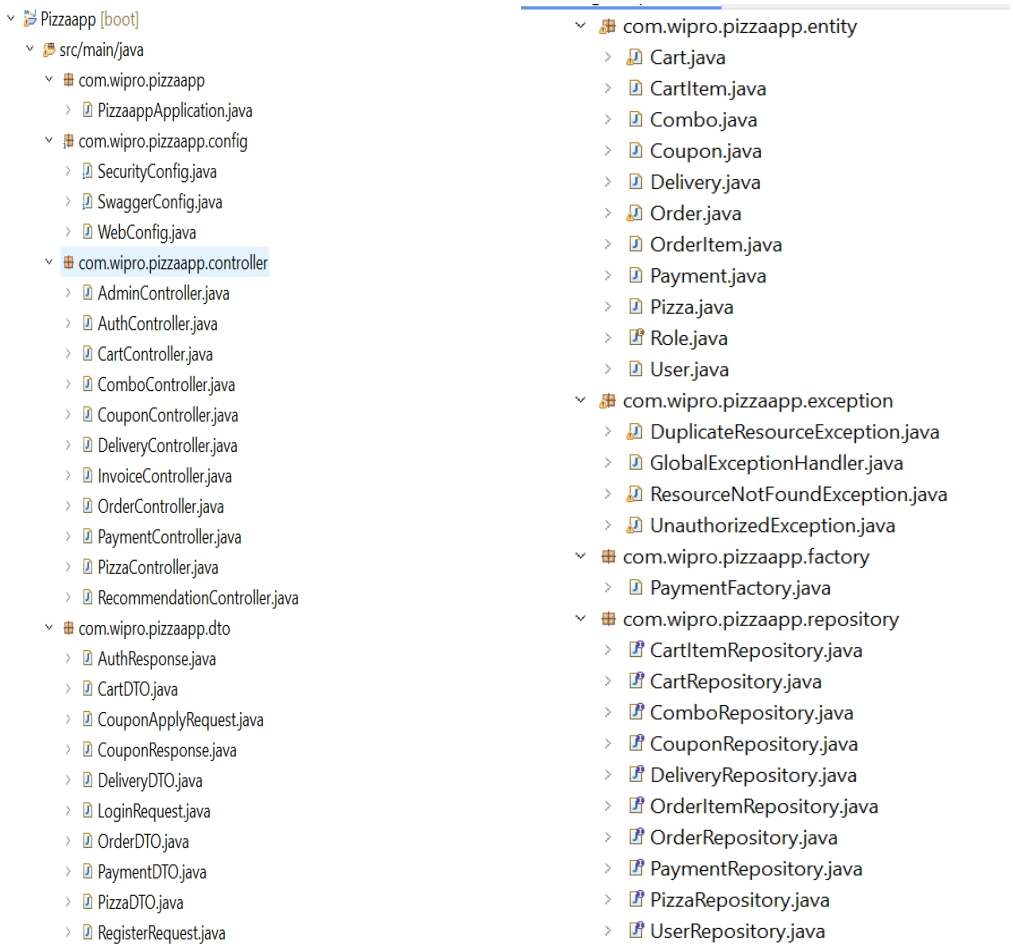
strategy

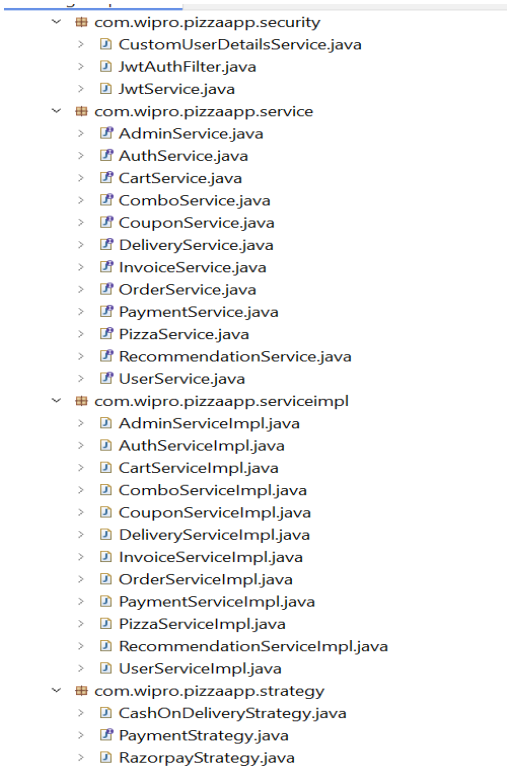
Implements the Strategy Design Pattern for payment processing.

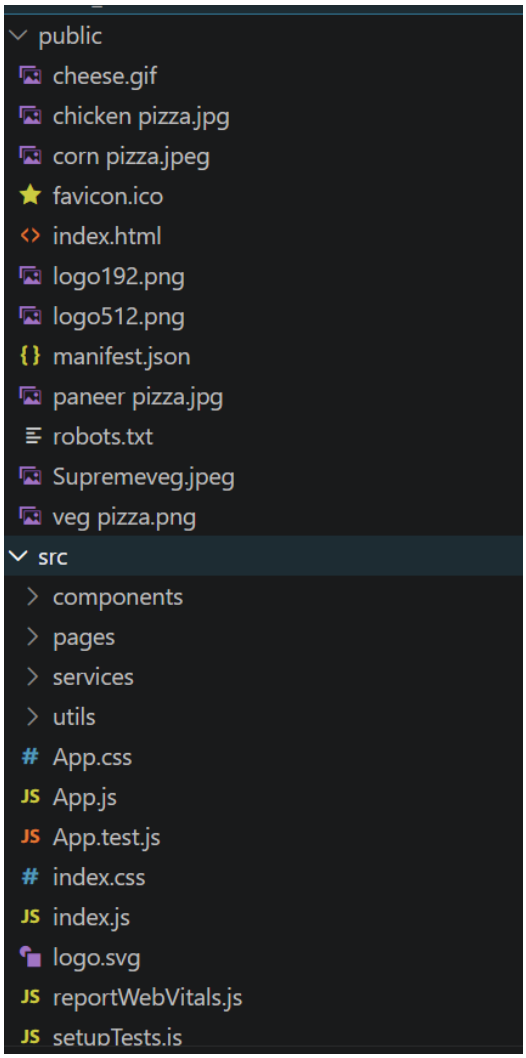
Classes:

- PaymentStrategy
- RazorpayStrategy
- CashOnDeliveryStrategy

This design enables flexible addition of new payment methods without modifying existing business logic.







Business Logic

The SmartPizza AI system follows a simple workflow to manage customers, orders, payments, and deliveries efficiently.

Authentication

Users can register and log in as Customer, Admin, or Delivery Partner. JWT authentication is used to provide secure access based on user roles.

Pizza Ordering

Customers can browse pizzas, view details, and add their favorite pizzas to the cart for ordering**.**

Cart Management

Customers can add, update, or remove items from the cart. The cart automatically calculates the total amount based on pizza price and quantity.

Coupon & Combo Management

Customers can apply valid coupons to receive discounts. Smart combo offers provide multiple pizzas at a reduced price.

Recommendation

The system recommends pizzas based on customer preferences, previous orders, and popular menu items to improve the ordering experience.

Order Processing

When an order is placed, the cart items are converted into order items, and the order status is marked as PLACED.

Payment Processing

Customers can pay using Razorpay. Successful payments update the order status accordingly.

Delivery Tracking

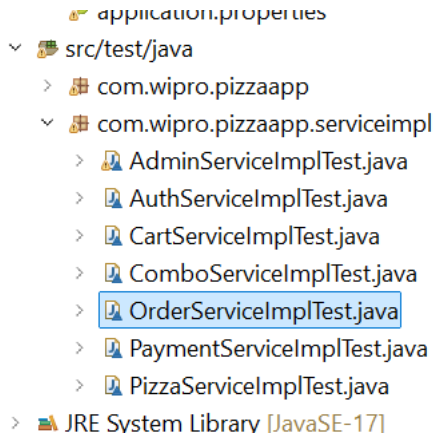
Each order is assigned a delivery record. Customers can track delivery progress through statuses such as Placed, Preparing, Out for Delivery, and Delivered.

Admin Analytics

Admins can manage pizzas, orders, coupons, and combos while monitoring key metrics such as total orders, revenue, customer activity, and delivery performance.

Testing Documentation

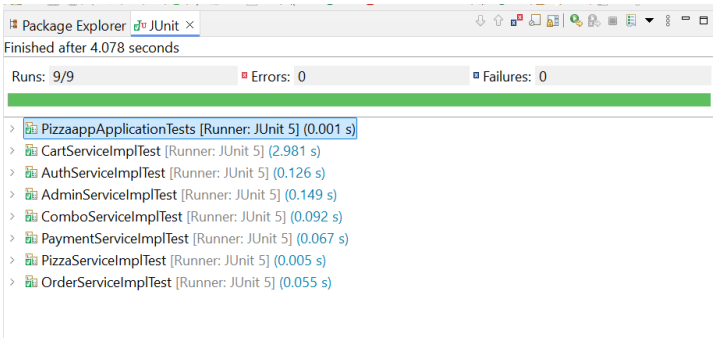
Unit Test Coverage



Test Cases

Test cases were created for authentication, cart management, order processing, payment integration, and delivery tracking. Each module was validated against expected and actual results.

Test Class	Module Tested	Test Scenario	Result
AdminServiceImplTest	Admin Module	Verify total orders and revenue statistics	Passed
AuthServiceImplTest	Authentication	User Registration and Login	Passed
CartServiceImplTest	Cart Management	Add items to cart and calculate total	Passed
ComboServiceImplTest	Combo Module	Create combo and calculate discounted price	Passed
OrderServiceImplTest	Order Module	Place order and verify status	Passed
PaymentServiceImplTest	Payment Module	Razorpay payment processing	Passed
PizzaServiceImplTest	Pizza Module	Retrieve pizza menu list	Passed



Execution Results

All major functionalities including login, ordering, payment processing, AI recommendations, and delivery tracking were executed successfully. The system met all functional requirements without critical failures.

Summary

SmartPizza AI – Intelligent Food Ordering & Delivery Optimization System is a web-based food delivery platform developed using React, Spring Boot, MySQL, JWT Authentication, and Razorpay Integration. The system enables customers to browse pizzas, receive AI-based recommendations, place orders, make secure payments, and track deliveries in real time.

The application supports Customer, Admin, and Delivery Partner roles with role-based access control and secure authentication. It includes features such as cart management, coupon engine, smart combo suggestions, AI recommendation engine, delivery tracking, ETA prediction, and online payment processing.

Administrators can manage pizzas, coupons, orders, users, and analyze business performance through an analytics dashboard, while delivery partners can update delivery status and manage assigned orders. The project improves customer experience, operational efficiency, and business decision-making through intelligent automation and data-driven insights.